Git & GitHub

Name: Shiqi Fang

November 22, 2024

1 Introduction

This document serves as a guide to Git and GitHub, highlighting their functionality and practical applications. Git, a version control system, enables tracking changes, reverting to previous versions, and facilitating distributed collaboration. GitHub, a cloud-based platform, enhances teamwork with features like real-time collaboration, project management tools, and public sharing. The guide covers essential commands for adding, modifying, and deleting files, managing branches, and synchronising local and remote repositories.

2 Applications

Figure 1: Current issues.

Current issues:

- Multiple files: Keeping all versions requires saving each version as a separate file
- Collaboration: Sharing files for teamwork often involves bundling and sending them manually
- Risk of loss: Deleted files may be permanently lost

3 Potential Solution

The potential solution involves the use of Git and GitHub.

3.1 Git

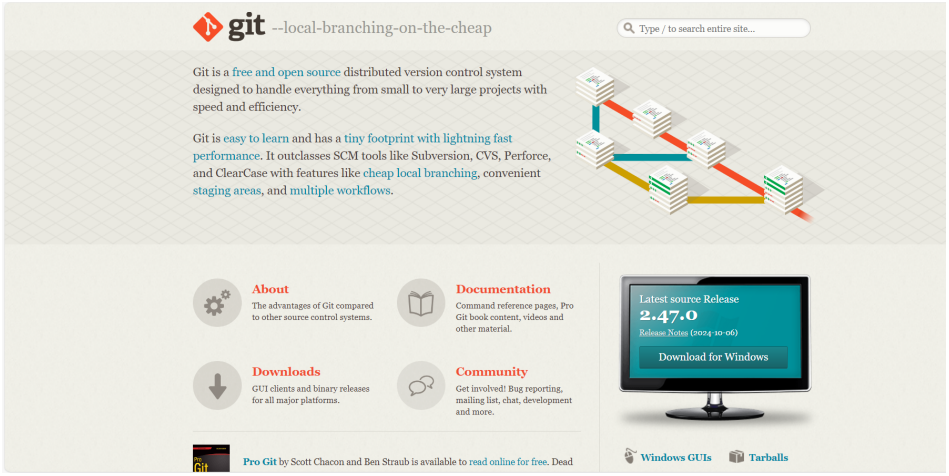


Figure 2: Git.

- Definition: Git is a Version Control System that saves versions of a project.
- Functions:
  - Tracks changes and enables reverting to previous versions.
  - Helps resolve errors and track project progress.
  - Allows distributed collaboration by enabling developers to push and pull changes across systems.

3.2 GitHub

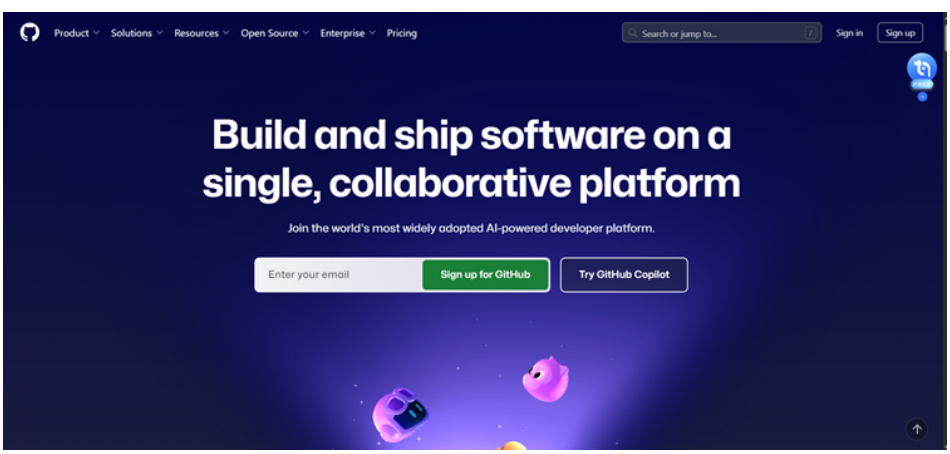


Figure 3: GitHub.

- Definition: GitHub is a cloud-based platform that enhances collaboration by hosting Git repositories
- Functions:
  - Facilitates real-time collaboration by allowing developers to look through and interact with each other's work.
  - Includes project management tools such as task assignment, permissions, and comment moderation.

- Enables public sharing and code contributions, fostering “social coding.”
- Provides a platform for developers to showcase their contributions and projects.

3.3 Connection between Git & GitHub

Git manages local repositories[1]; GitHub connects them to the cloud for distributed teams to collaborate.

Changes made in local repositories can be pushed to GitHub via Git, and updates from others can be pulled back into local systems.

4 Download

- Download Git

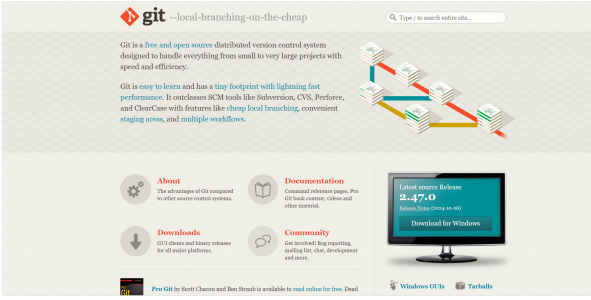


Figure 4: Git.

The second is to use Git from the CMD and third-party software, not limited to Git Bash.

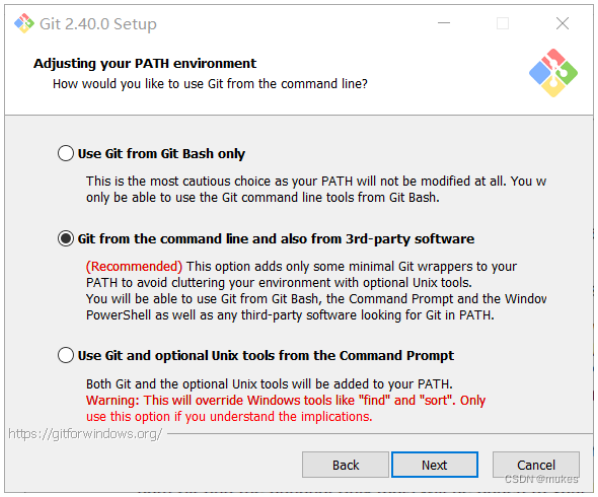


Figure 5: Download Git.

- Initialise the setup for Git

[1]Local Repository: A local repository is the copy of a Git repository stored on your local machine. It contains all the project’s files, commit history and the .git directory that tracks changes. Developers work directly with the local repository to make changes, stage, and commit files before pushing them to a remote repository.

Remote Repository: A remote repository is a version of the repository hosted on a remote server or a platform like GitHub, GitLab, or Bitbucket. It serves as a centralized location for collaboration, allowing multiple users to fetch, pull, and push changes to maintain a synchronized project across different environments.

To allow the remote repository to identify who made the commit, Git requires a username and email address.

Configuration commands:

```
git config --global user.name "Your Name"
git config --global user.email "email@example.com"
```

Note:

- It is recommended to use a real email address associated with GitHub for proper identification.
- If the email address is linked to a GitHub account, the commits will display the corresponding GitHub account.
- If the email is not linked to GitHub, the configured username will appear in the commits.

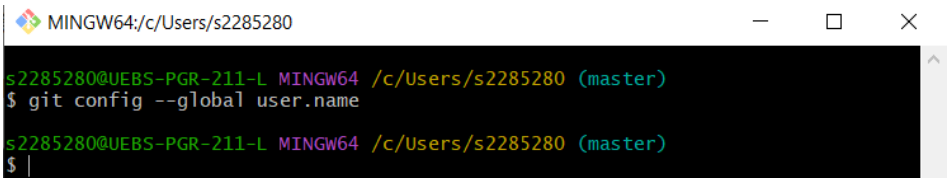


Figure 6: git config –global user.name without an answer.

Figure 7: git config –global user.name with an answer after resetting.

- Register an account for Github

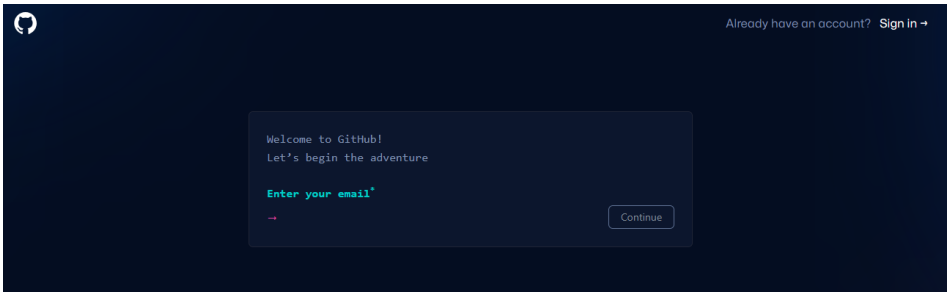


Figure 8: Registration for GitHub.

- Connect git to GitHub: Create a repository in GitHub

  Step 1: fill out the information for your remote repository

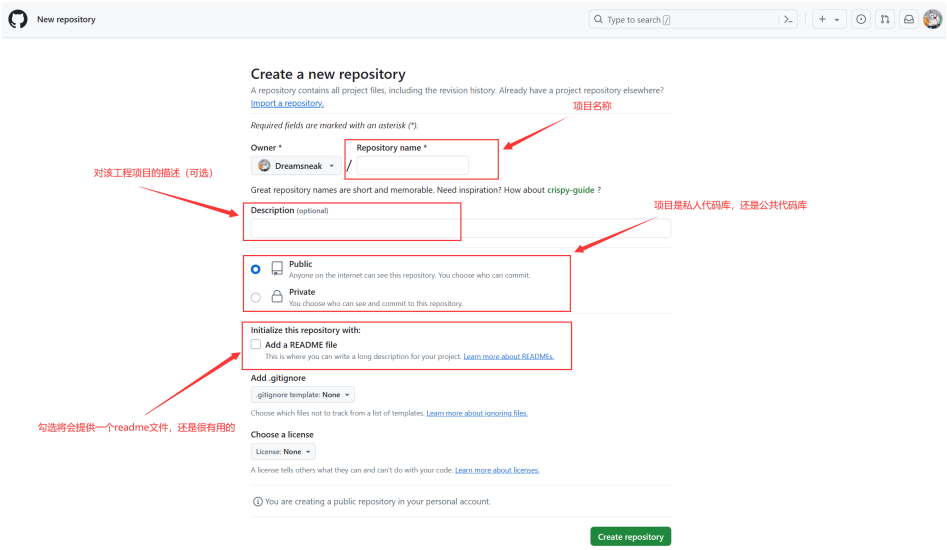


Figure 9: Create a repository-Step 1.

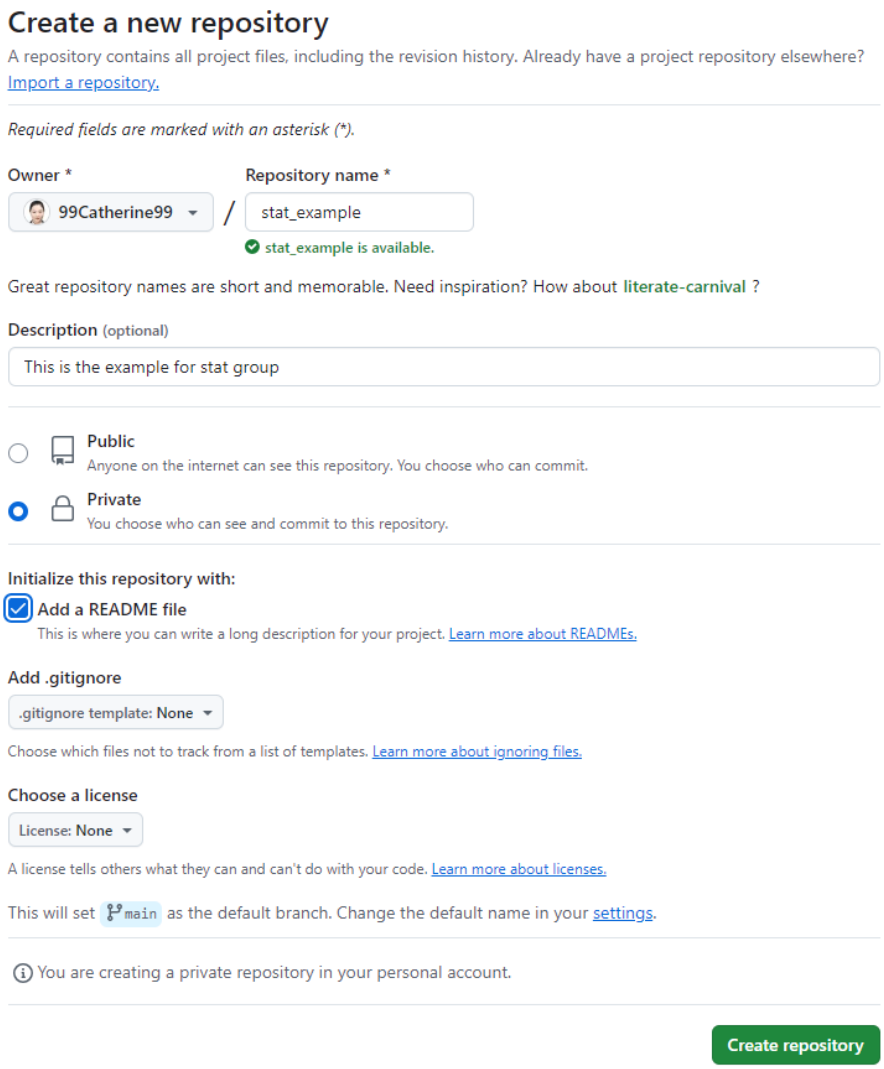


Figure 10: Create a repository-Step 1.

Step 2: check HTTPS for your remote repository

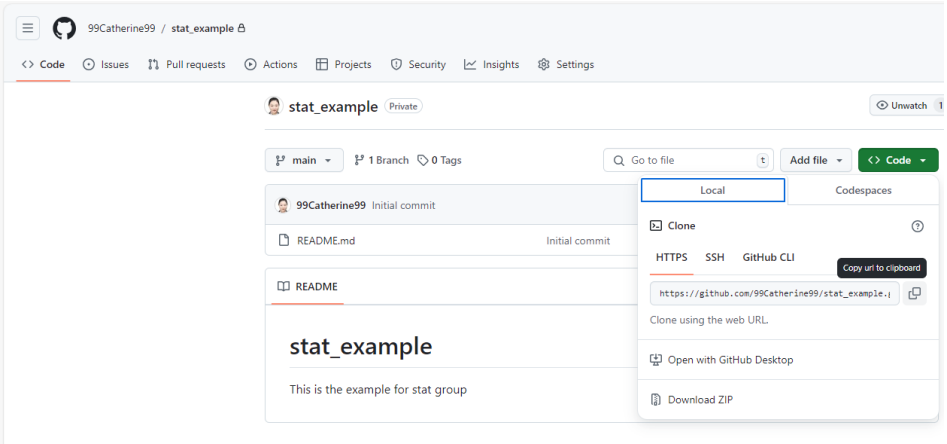


Figure 11: Create a repository-Step 2.

5 Commands

5.1 Clone a repository

Clone the remote repository to your local, then you will have the local repository. The connection between Git and GitHub begins!

```
git clone https://github.com/99Catherine99/stat_example.git
```

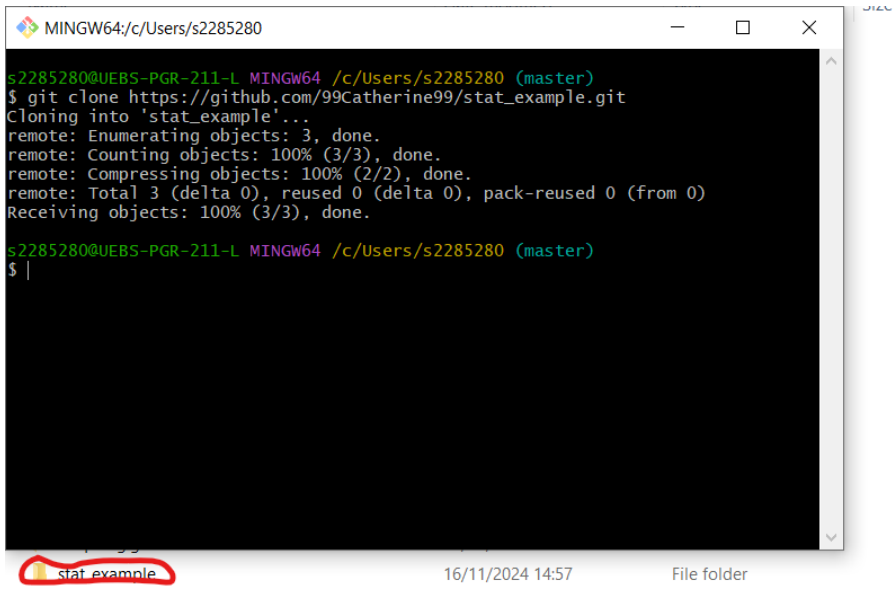


Figure 12: Clone repository to the local.

5.2 Check the information under your local repository

```
git config user.name
git config user.email
```

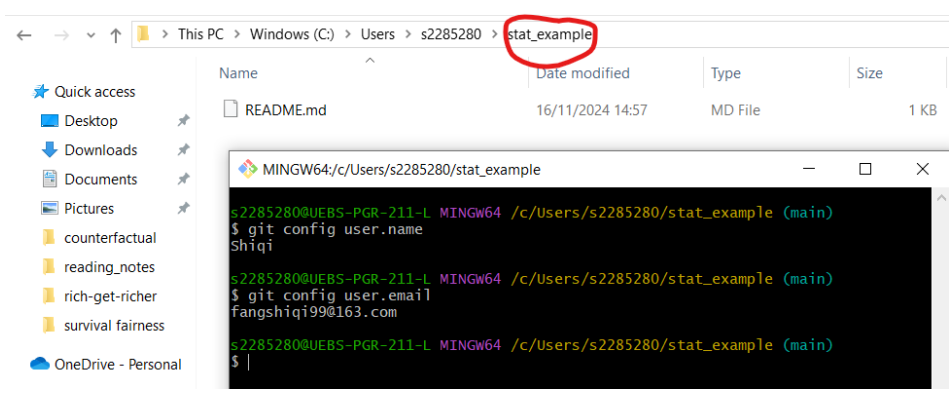


Figure 13: Check information under the local repository.

5.3 Add, modify, delete files

The Workflow

1. Stage Changes: Use 'git add', 'git mv', 'git rm'..... to move files or changes into the staging area. Only staged files are included in the next commit.
2. Commit Changes: Use git commit to save the staged changes into the local repository.
3. Push Changes: Use git push to send the committed changes to the remote repository.

How could we map the changes to the remote repository if we add, modify or delete files in our local repository?

- `git add file1`
- `git add -u`
- `git add -A`
- `git add .`
- `git add *`

Command	Scope	Behaviour
`git add file1`	Specific file	Stages only the specified file.
`git add -u`	Current directory and subdirectories	Stages only changes to tracked files (modified and deleted files). Doesn't stage untracked files.
`git add -A`	Entire repository	Stages all changes (new, modified, and deleted files) in the entire repository, including hidden files.
`git add .`	Current directory and subdirectories	Stages new and modified files, including hidden files.
`git add *`	Current directory only	Stages new and modified files but ignores hidden files and subdirectories.

Table 1: Comparison of `git add` commands.

5.3.1 Add specific file

If we only add a specific file to the local repository.

```
git add file1
```

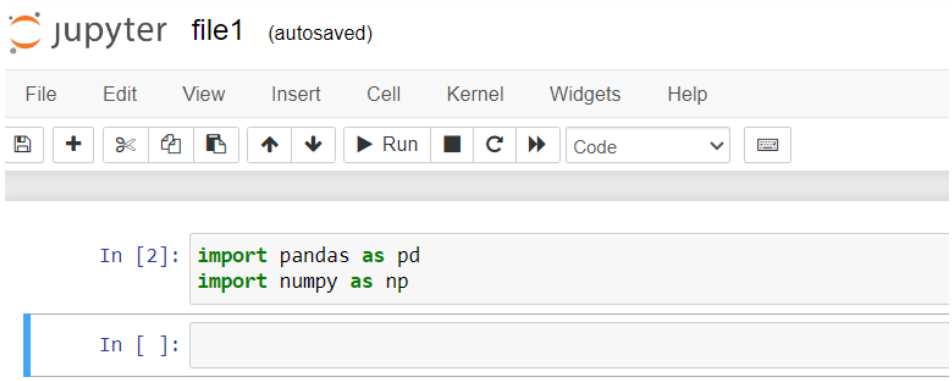


Figure 14: Modify file1.

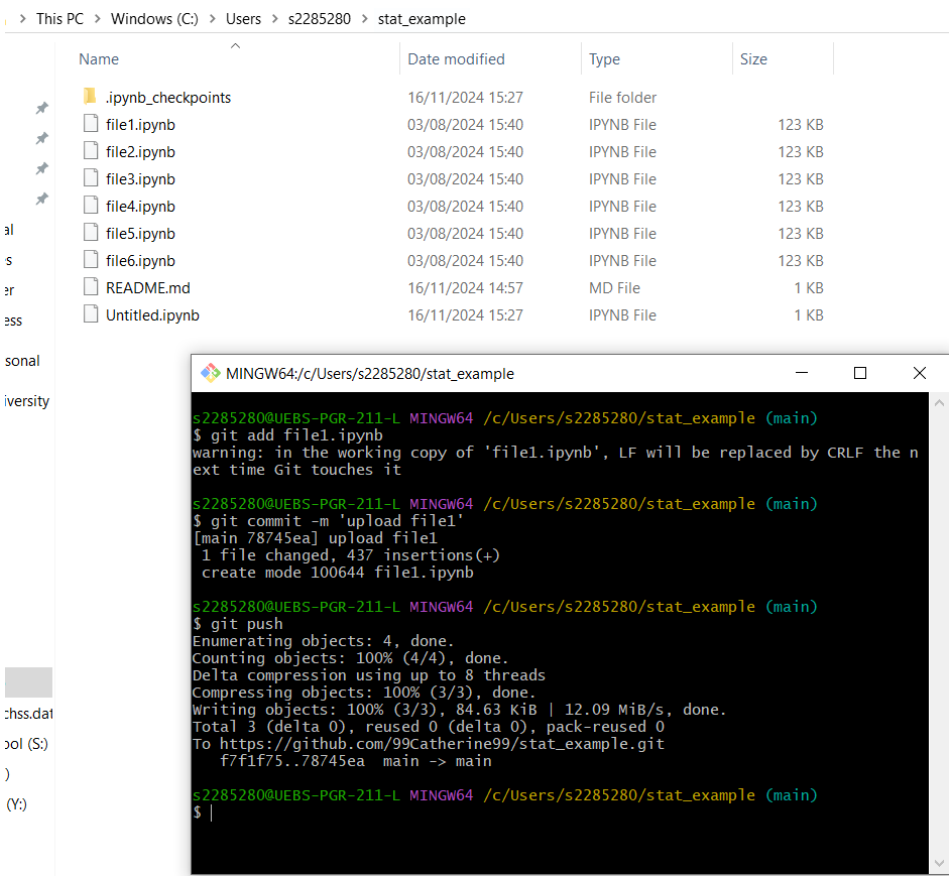


Figure 15: Add file1.

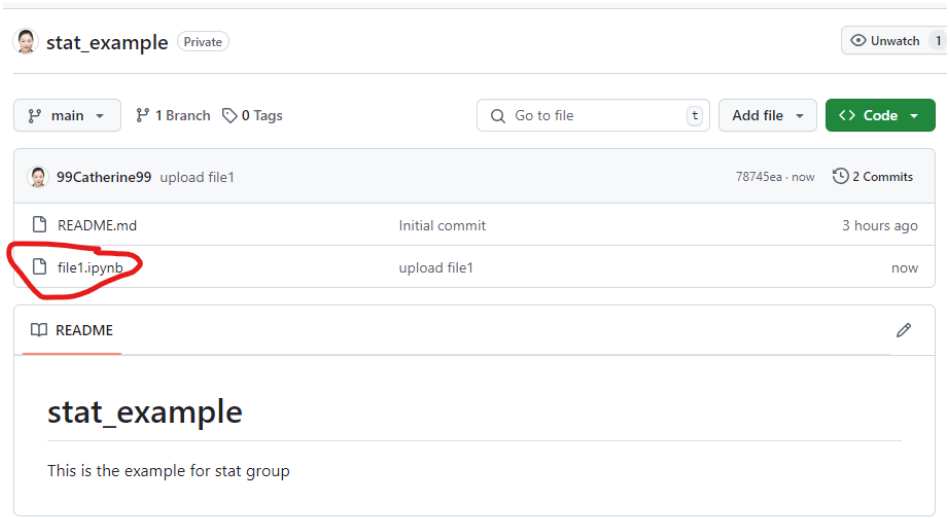


Figure 16: File1 in the remote.

5.3.2 Add tracked files

If we want to map only the tracked[2] files to the remote.

```
git add -u
```

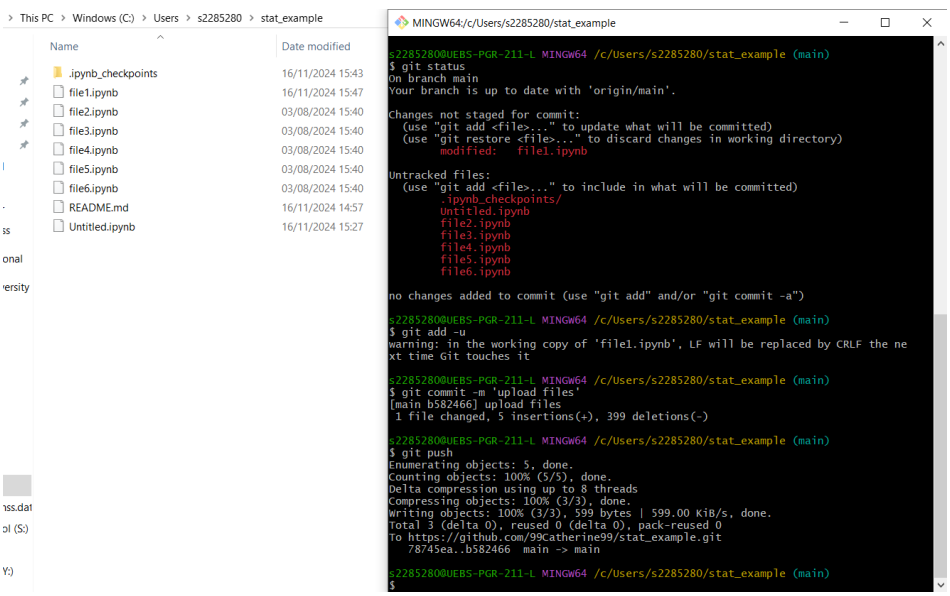


Figure 17: Add tracked files.

5.3.3 Add all files

If we want to map the changes to the remote, including modified, hidden, under the sub dictionary, and untracked, we need to apply:

[2]Tracked files in Git are files that are already under version control. These include files that have been added to the staging area using git add and subsequently committed to the repository. Tracked files can be in one of three states: modified, staged, or unmodified. In contrast, files not yet added to Git are referred to as untracked files.

```
git add -A
```

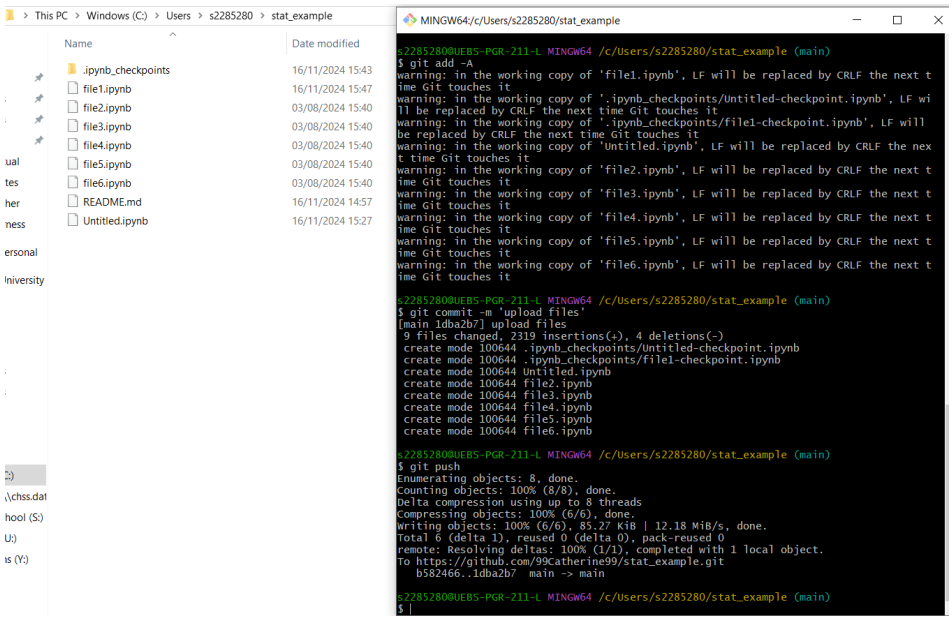


Figure 18: Add all files.

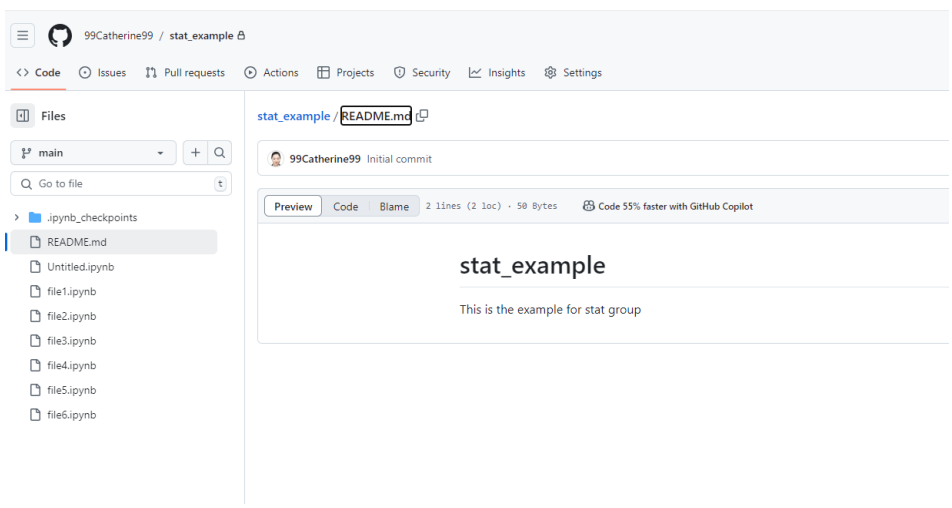


Figure 19: All files in the remote.

5.3.4 git add .

In earlier versions of Git, 'git add .' did not stage removed files for the next commit. However, the latest version now includes removed files in the staging area as well. It is now similar to 'git add -A'

```
git add .
```

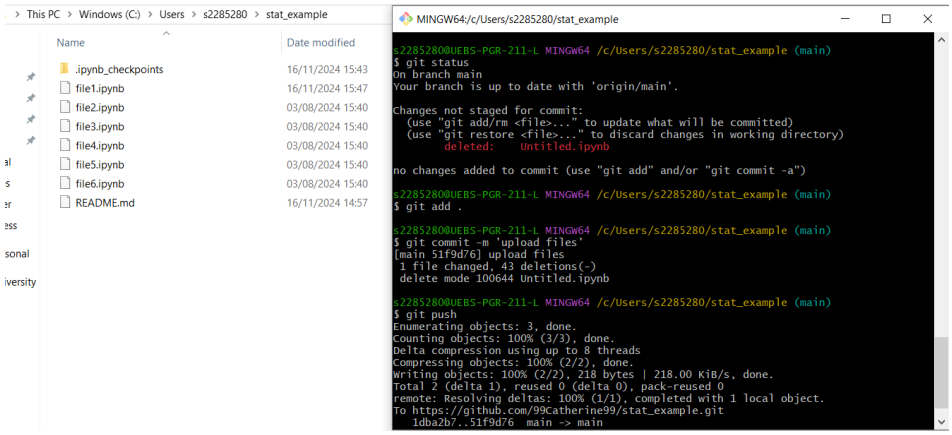


Figure 20: Add all files.

5.3.5 Add files which are not hidden files or in the Subdirectories

```
git add *
```

5.4 Delete files

If we delete a specific file:

```
git rm file1
```

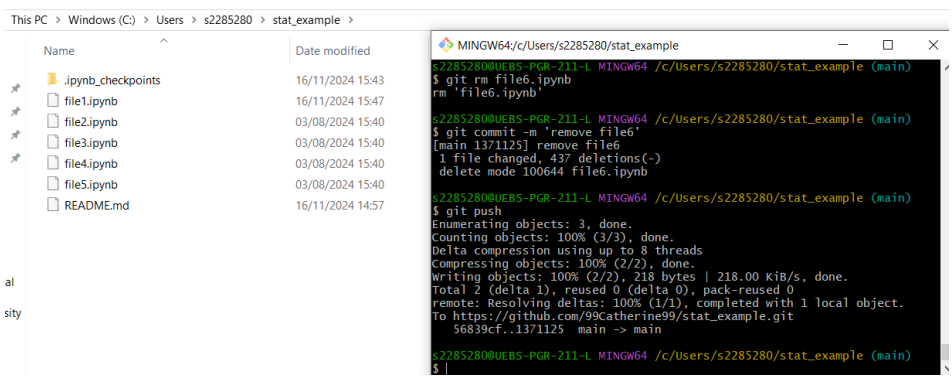


Figure 21: Delete file1.

Note:
We could also use 'git add .' or 'git add -A' to map the changes to the remote since these commands could also track the removed files.

5.5 Add folders

```
mkdir foder1
```

But commit could only record and track the folder with files. The foder1 is now an empty one, so it shows everything up-to-date after 'git push'.

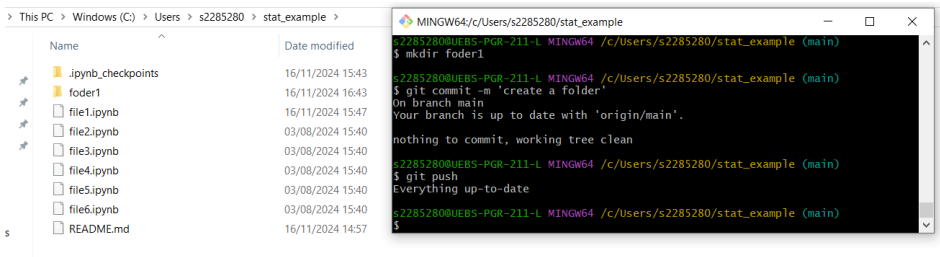


Figure 22: Create folder.

We add files to the folder and commit, then we could update the changes in the remote. We could see that there is foder1 now in GitHub.

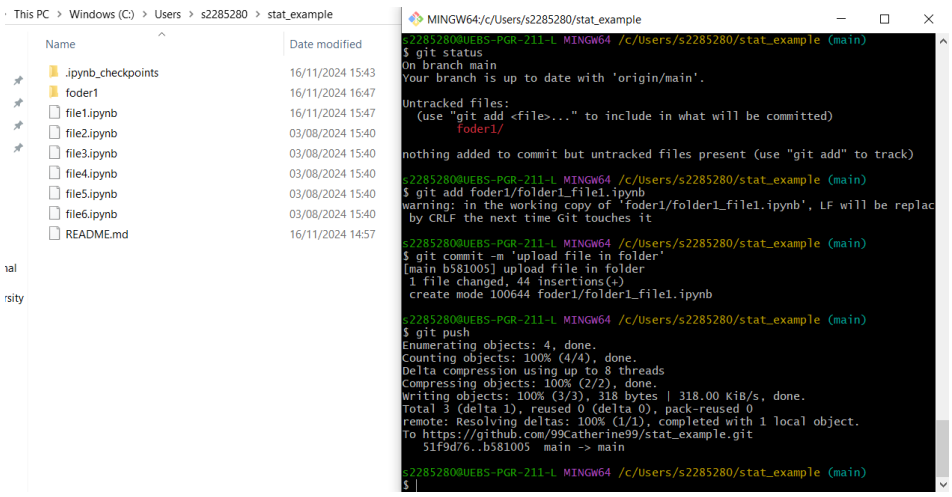


Figure 23: Push non-empty folder to the remote.

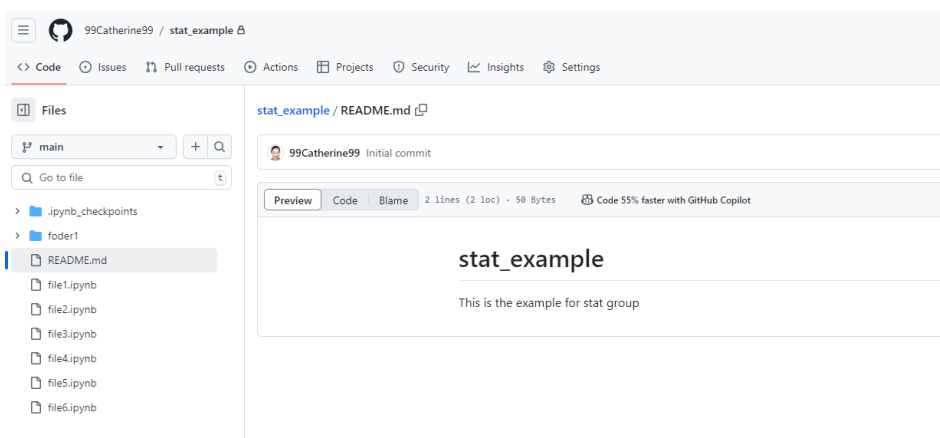


Figure 24: Folder in the remote.

5.6 Change the name of the folder

foder1 should be folder1, so we need to change the name of our folder and then commit and push.

```
git mv foder1 folder1
```

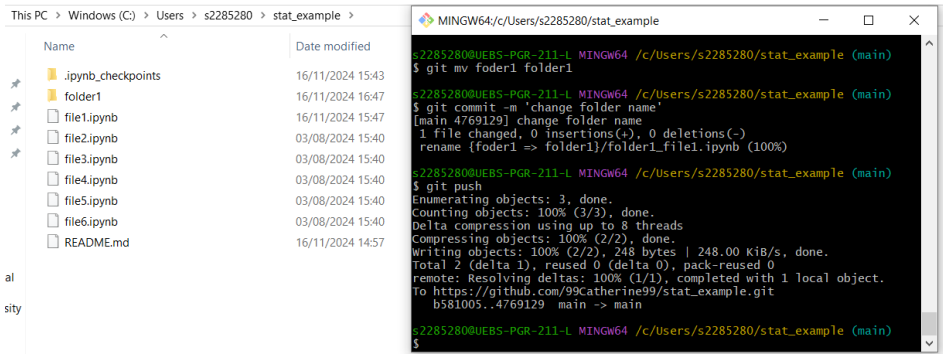


Figure 25: Change the name of folder.

5.7 Delete the file in the folder

```
git rm folder1/folder1_file1
```

5.7.1 If there is only file1 under folder1

If there is only file1 under folder1, if we remove the file, then the folder will also be removed.

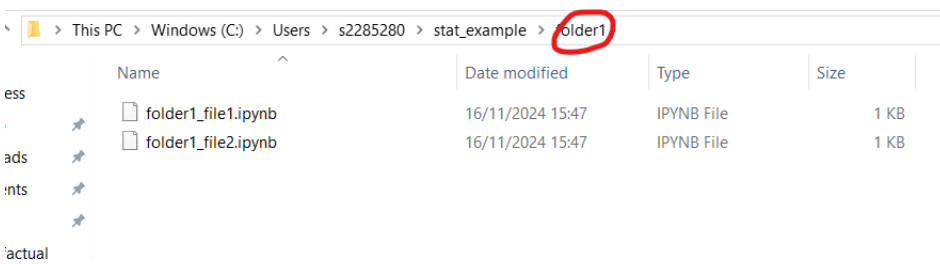


Figure 26: Situations in the folder before command.

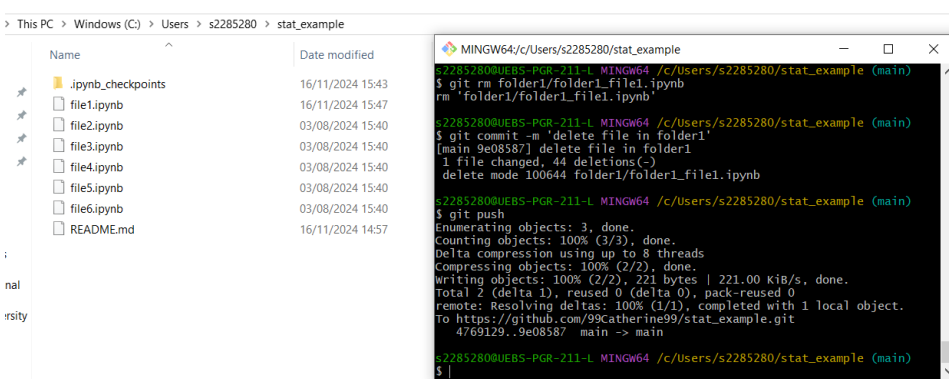


Figure 27: Delete the only file in a folder.

5.7.2 If there are other files

If there are files other than file1, then the folder will still exist.

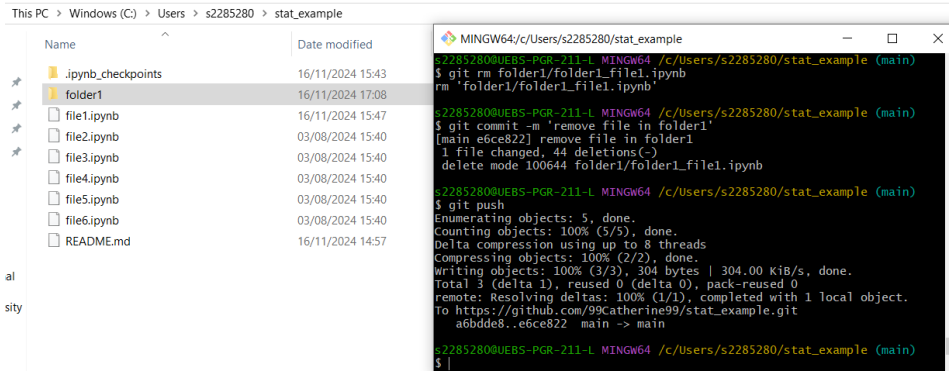


Figure 28: Delete non-unique files in a folder.

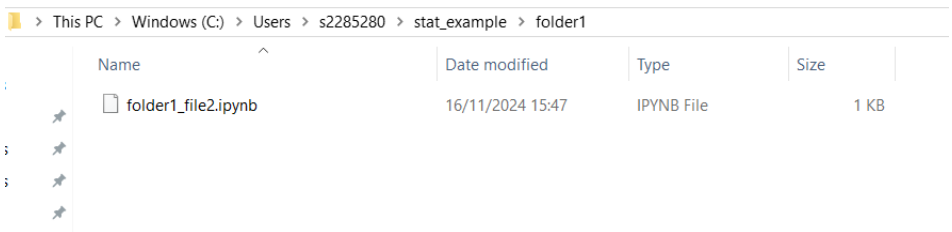


Figure 29: Situations in the folder after command.

5.8 Delete folder

```
git rm -r folder1
```

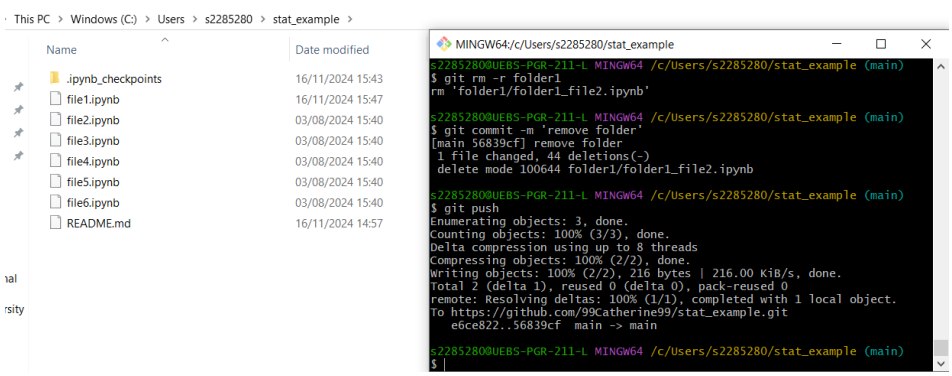


Figure 30: Delete folder.

5.9 Move files and folders

5.9.1 Move into folder

Move files into folder

```
git mv file5.ipynb folder1/
```

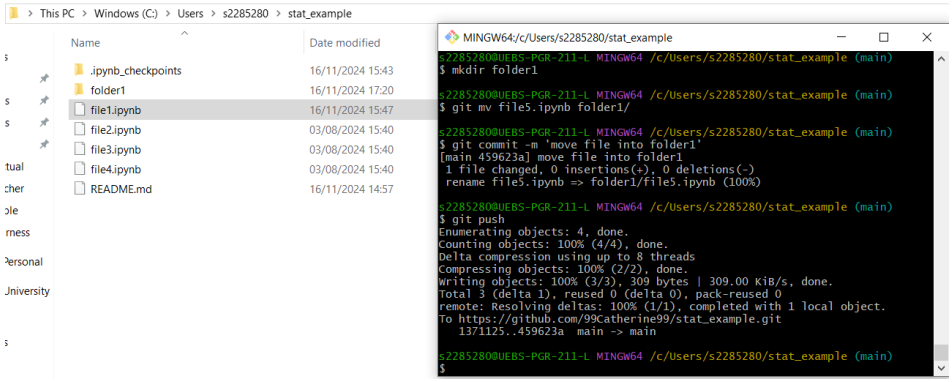


Figure 31: Move files into folder.

Move folders to folder

```
git mv folder2 folder1/
```

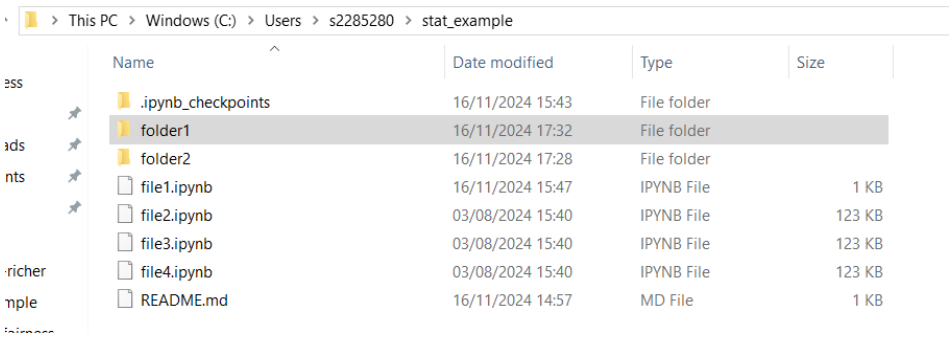


Figure 32: Situations before command.

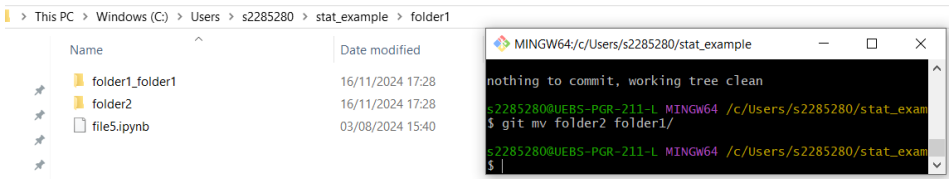


Figure 33: Move folders into folder.

5.9.2 Move out of folder

Move files out of folder

```
git mv folder1/file5 .
```

Move folders out of folder

```
git mv folder1/folder2 .
```

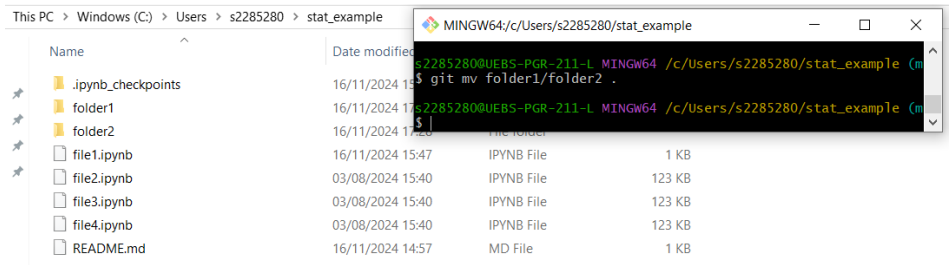


Figure 34: Move folders out of folder.

5.10 commit

Enter the commit message directly in the command.

```
git commit -m 'updating file'
```

5.11 push

The command is used to upload commits in the local repository to the remote repository. It will synchronize the contents of the local branch to the remote branch.
We will also introduce other variants in the next section.

```
git push
```

5.12 log

We will show it in the next section.

```
git log
```

5.13 diff

We will show it in the next section.

```
git diff
```

6 Branch

6.1 Main, Master, Branch

Concept	Description	Purpose
Branch	An independent line of development in Git.	Used for developing new features, fixing bugs, or testing changes without affecting the main codebase.
Master	The traditional default branch name in Git.	Typically used for production-ready and release versions.
Main	The modern default branch name in Git.	Functions the same as `master`.

Table 2: Comparison of Branch, Master, and Main in Git

6.2 List branch

check the current branch: the asterisk (*) marks the current branch.

```
git branch
```

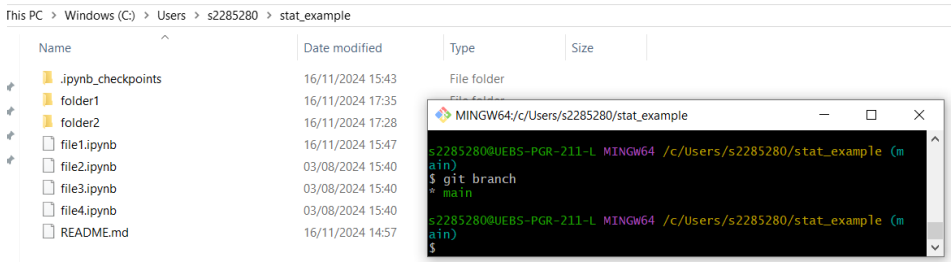


Figure 35: Check the current branch.

list all the branches

```
git branch -r
```

6.3 Create and switch to branches

create a branch and name it $first_branch$

```
git branch first_branch
```

switch to the branch

```
git checkout first_branch
```

create and get into the branch

```
git checkout -b first_branch
```

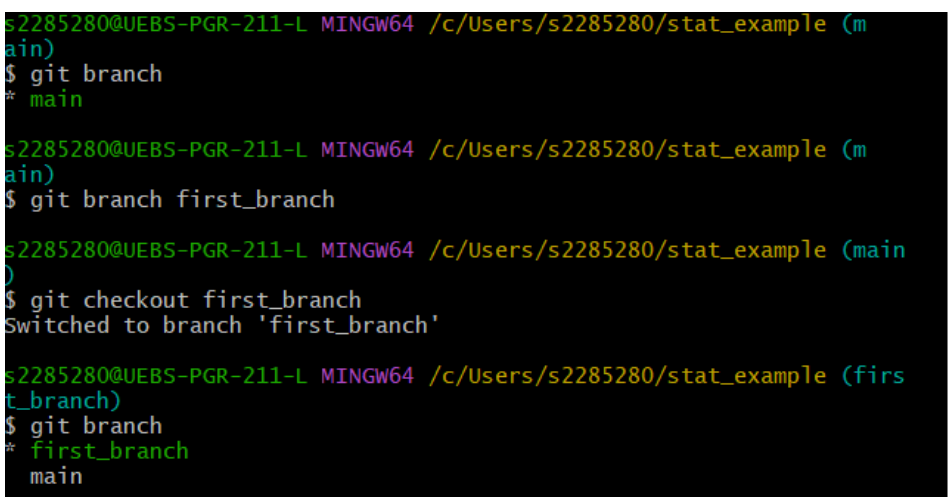


Figure 36: Create and switch to branch.

6.4 Push to the remote

```
git push -u origin first_branch
```

After switching to the branch, we modify the README.md in this branch, and then commit and push to the remote (i.e., origin[3]).
README.md in this branch:

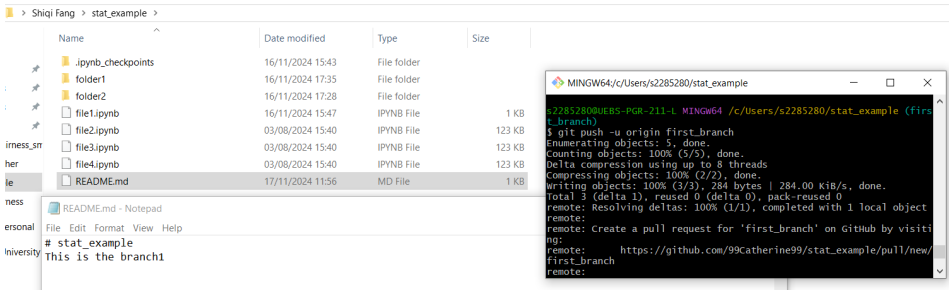


Figure 37: Push the changes in the branch to the remote.

Note: Push the specified branch to the remote repository and set the remote branch as the default upstream branch to simplify the subsequent use of git push operations. Therefore, if this is the first time to push this branch, then apply:

```
git push -u origin first_branch
```

For the following push, we could apply:

```
git push origin first_branch
```

We could see in the GitHub remote, that the README.md in the branch is like this:

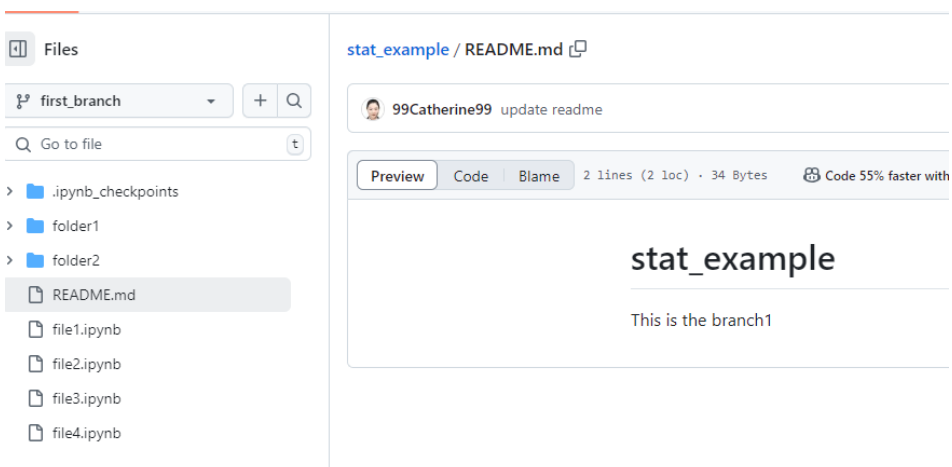


Figure 38: README.md in the branch.

While README.md in the main is like:

[3]origin in Git: The default name for the remote repository created when you clone a repository. It serves as an alias for the repository's URL.

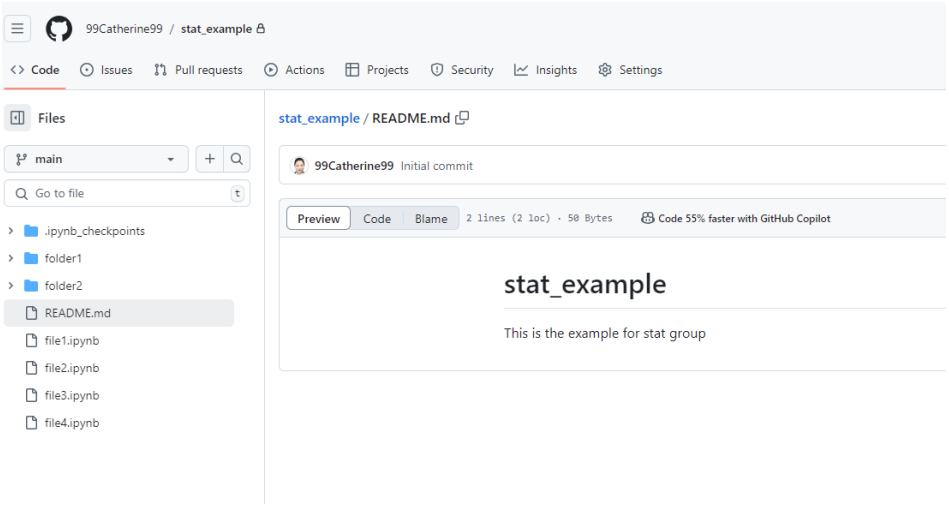


Figure 39: README.md in the main.

6.5 Pull

Due to the difference between the main and the branch, GitHub shows merge information, it is asking if you want to merge the difference to the main.

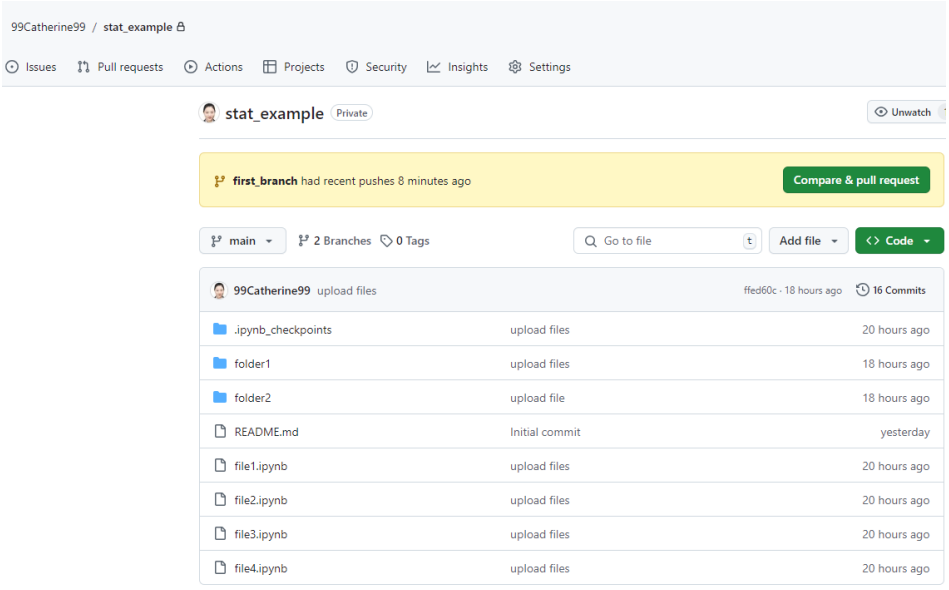


Figure 40: Merge request the difference between the branch and main.

We could check the difference there

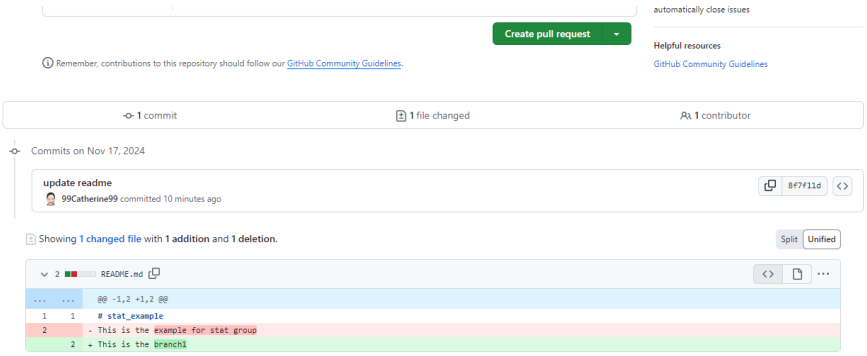


Figure 41: Difference information between the branch and main.

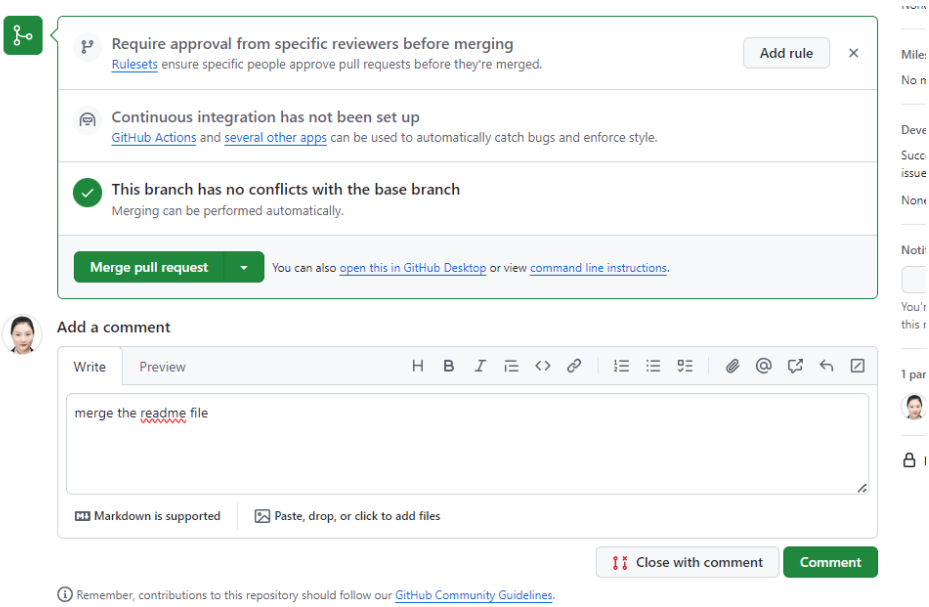


Figure 42: Confirmation of merge.

After confirming merging, README.md in the main is

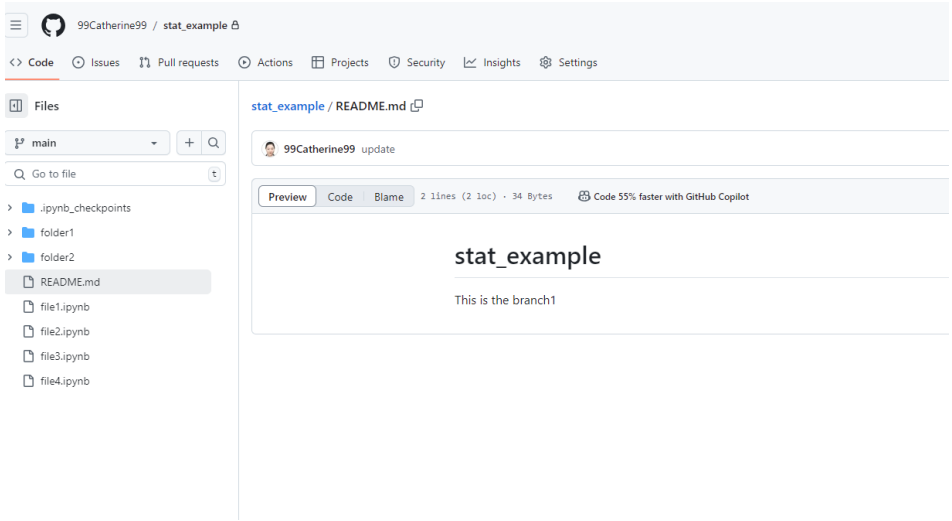


Figure 43: README.md in the main after marge confirmation.

However, our local repository has not changed, we use the command to update the changes: pull all the updates from the remote to the local repository.

```
git pull
```

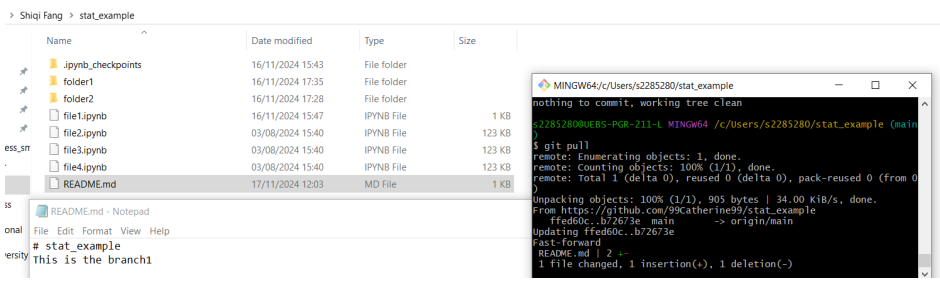


Figure 44: Pull from the remote to the local.

6.6 Create and switch into the second branch

```
git checkout -b second_branch
```

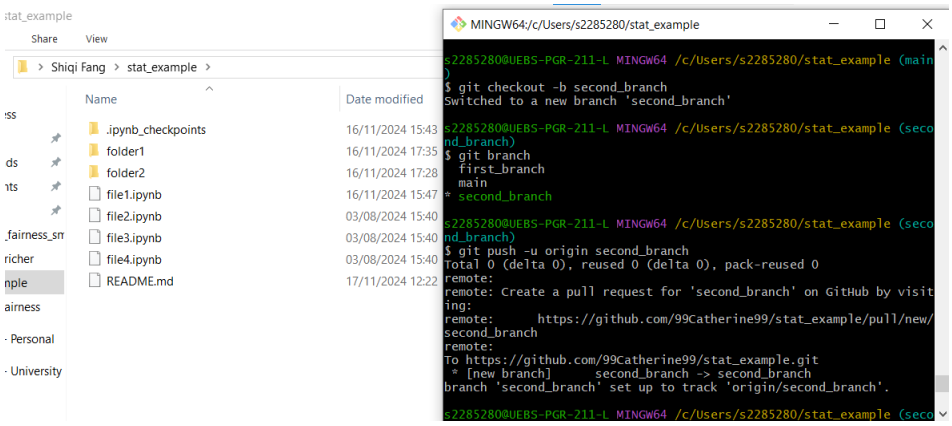


Figure 45: Create and switch to the other branch.

6.7 Download changes without merge

download the update to the local, not merge it with the main

```
git fetch origin
```

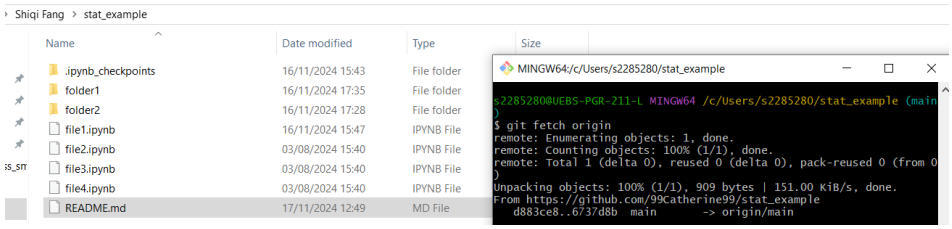


Figure 46: Download changes without merge.

6.8 Check the updated history and the difference within two repositories

```
git log
```

```
git diff
```

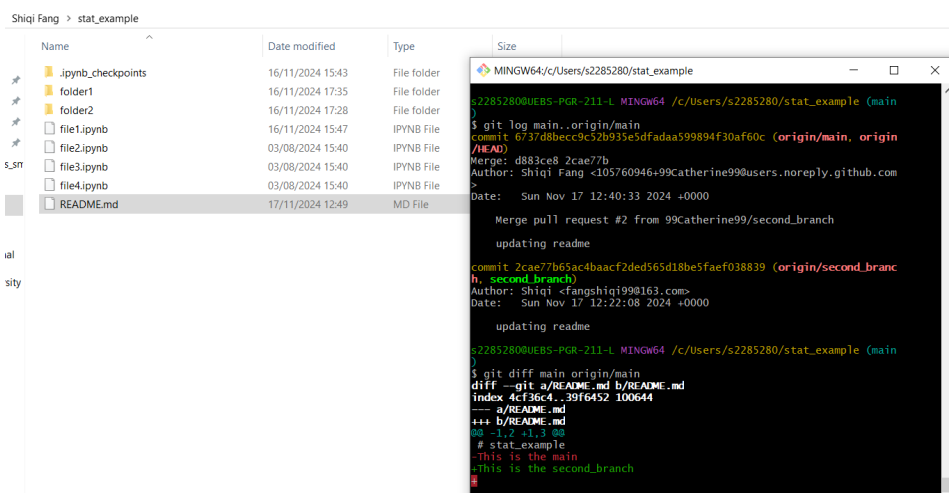


Figure 47: Check the updated history and the difference within two repositories.

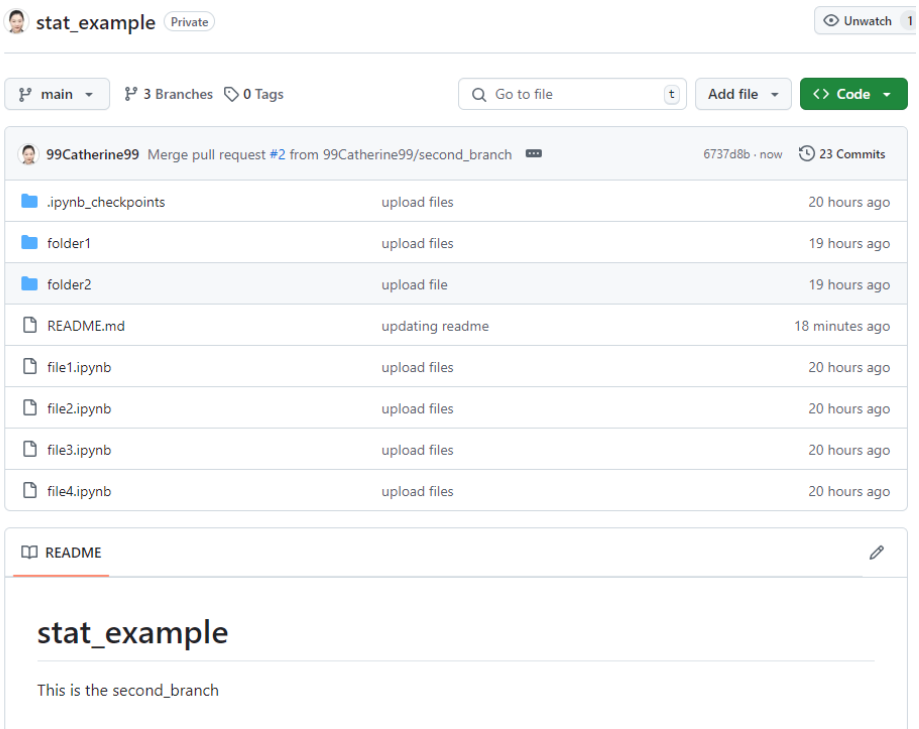


Figure 48: README.md in the main in the remote.

6.9 Merge the difference

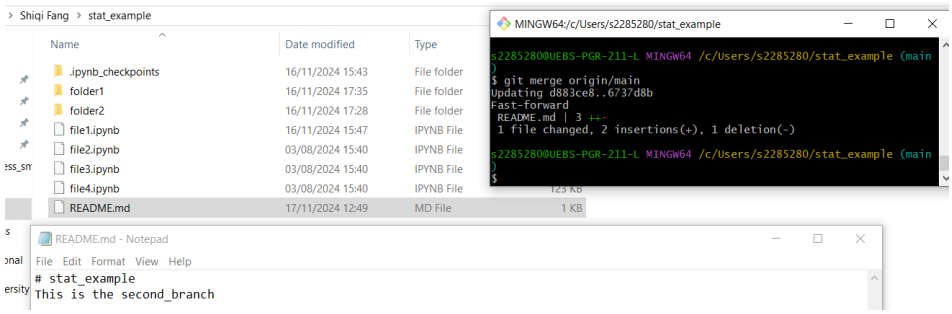


Figure 49: Merge the difference.

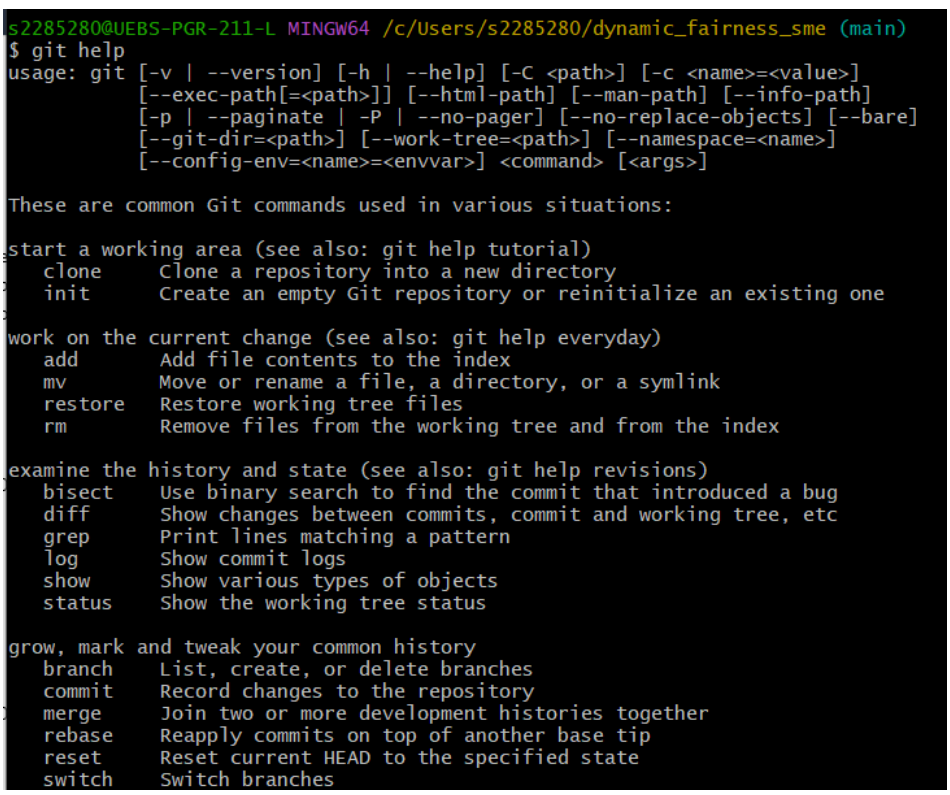
Command	Purpose	Key Difference
`git pull`	Fetches changes from the remote repository and merges them into the current branch.	Combines `git fetch` and `git merge` in a single step. Automatically updates your working directory.
`git fetch`	Downloads changes from the remote repository but does not apply them to your working directory.	Fetches updates without affecting the current branch. Useful for reviewing changes before merging.
`git merge`	Combines the changes from one branch into another, typically integrating fetched changes into the current branch.	Requires `git fetch` beforehand if merging remote changes. Used to manually control the integration process.

Table 3: Comparison of `git pull`, `git fetch`, and `git merge`

6.10 Find help

You will find help from

```
git help
```



```
s2285280@UEBS-PGR-211-L MINGW64 /c/Users/s2285280/dynamic_fairness_sme (main)
$ git help
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
           [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
           [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--bare]
           [--git-dir=<path>] [--work-tree=<path>] [--namespace=<name>]
           [--config-env=<name>=<envvar>] <command> [<args>]

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
   clone     Clone a repository into a new directory
   init      Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)
   add       Add file contents to the index
   mv        Move or rename a file, a directory, or a symlink
   restore   Restore working tree files
   rm        Remove files from the working tree and from the index

examine the history and state (see also: git help revisions)
   bisect    Use binary search to find the commit that introduced a bug
   diff      Show changes between commits, commit and working tree, etc
   grep      Print lines matching a pattern
   log       Show commit logs
   show      Show various types of objects
   status    Show the working tree status

grow, mark and tweak your common history
   branch    List, create, or delete branches
   commit    Record changes to the repository
   merge     Join two or more development histories together
   rebase    Reapply commits on top of another base tip
   reset     Reset current HEAD to the specified state
   switch    Switch branches
```

Figure 50: Find help.

7 CMD

You might use CMD to implement

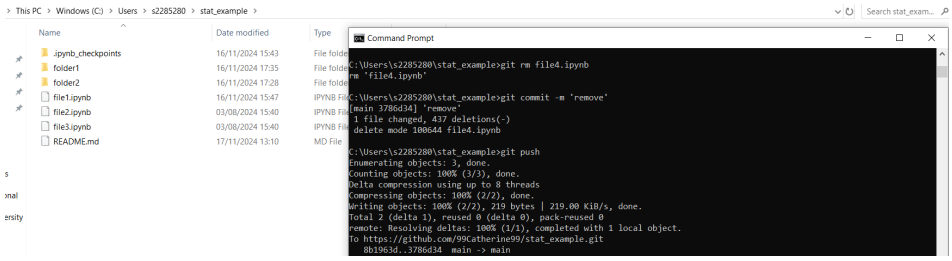


Figure 51: CMD for Git.

8 Others

- reset, revert, restore
- conflicts between branches

9 Link

Links for commands: Link1 & Link2
Links for Mac: Download for Mac & Git for Mac